

Entendendo o Script

CarController.cs

Um guia comentado, parte por parte, de um controlador de carro em C#
para Unity

Feito para acompanhar o desenvolvimento do seu projeto no Unity

Sumário

1. Visão geral do script
2. Cabeçalho e declaração da classe
3. Variáveis configuráveis no Inspector
4. Variáveis internas de controle
5. Referência ao Transform do carro
6. Update() — o coração do script
7. Atualizar() — organizando a ordem das coisas
8. AtualizarInput() — lendo o jogador
9. AtualizarFisica() — calculando a velocidade
10. AplicarMovimento() — movendo e girando o carro
11. Getters — expondo dados para outros scripts
12. Glossário rápido de termos do Unity/C#

1. Visão geral do script

Esse script é um **controlador de carro (CarController)** para Unity, escrito em C#. Ele é responsável por três coisas, nessa ordem, a cada frame do jogo:

- **Ler o input** do jogador (teclado ou controle);
- **Calcular a física** simplificada de aceleração, freio e atrito;
- **Mover e girar** o objeto do carro na cena, de acordo com esses cálculos.

O script é anexado como **componente** a um GameObject no Unity (o carro), e o Unity chama automaticamente o método `Update()` a cada frame renderizado.

2. Cabeçalho e declaração da classe

```
using UnityEngine;
public class CarController : MonoBehaviour
```

Trecho 1 — início do script

- `using UnityEngine;` importa a biblioteca principal do Unity, que dá acesso a classes como `Transform`, `Vector3`, `Input` e `Debug`. Sem essa linha, nada do resto do script funcionaria.
- `public class CarController : MonoBehaviour` cria a classe do script. O `MonoBehaviour` é o que transforma esse código em um **componente do Unity** — ou seja, algo que pode ser arrastado para um GameObject no Inspector. Toda classe que herda de `MonoBehaviour` ganha acesso a métodos especiais que o Unity chama automaticamente, como `Update()` e `Start()`.

3. Variáveis configuráveis no Inspector

```
[Header("Velocidade")]
[SerializeField] private float velocidadeMaxima = 65f;
[SerializeField] private float velocidadeMaximaRe = 28f;

[Header("Direção")]
[SerializeField] private float anguloMaximoDirecao = 37f;
```

Trecho 2 — variáveis ajustáveis pelo Inspector

- `[Header("Velocidade")]` não é código de execução — é um **atributo** que só serve para organizar visualmente o Inspector do Unity, criando um título de seção.
- `[SerializeField]`: normalmente, variáveis `private` não aparecem no Inspector. Esse atributo força a variável a aparecer mesmo sendo privada — assim dá para ajustar o valor pela interface do Unity, sem mexer no código, mas outros scripts continuam sem poder acessá-la diretamente.
- `velocidadeMaxima = 65f`: velocidade máxima que o carro pode atingir indo para frente (o `f` no final indica que é um `float`, número decimal).
- `velocidadeMaximaRe`: limite de velocidade dando marcha à ré.
- `anguloMaximoDirecao`: o quanto as rodas podem girar no máximo (em graus) — o ângulo máximo de esterço.

4. Variáveis internas de controle

```
private float velocidadeAtual = 0f;
private float anguloDirecao = 0f;
private float anguloAlvo = 0f;
private float entradaAceleracao = 0f;
private float entradaDirecao = 0f;
private readonly Vector3 direcaoFrente = Vector3.zero;
```

Trecho 3 — variáveis internas (não aparecem no Inspector)

- `velocidadeAtual`: guarda a velocidade real do carro naquele exato momento, que muda a cada frame.
- `anguloDirecao`: o ângulo atual das rodas, que vai variando suavemente até chegar no `anguloAlvo`.
- `anguloAlvo`: para onde o jogador está tentando virar (o "objetivo" do ângulo).
- `entradaAceleracao` e `entradaDirecao`: guardam o que o jogador está apertando no momento (teclado ou controle), com valores entre -1 e 1.
- `direcaoFrente`: essa variável, na prática, não é usada em nenhum lugar do código (sobrou de uma versão anterior). Pode ser removida sem problema.
- `readonly`: indica que o valor só pode ser definido na declaração ou no construtor, nunca muda depois disso.

5. Referência ao Transform do carro

```
public Transform carroTransform;
```

Trecho 4 — referência pública ao Transform

- Transform é o componente do Unity que guarda **posição, rotação e escala** de um objeto na cena.
- Como é `public`, essa variável aparece no Inspector, e é preciso arrastar manualmente o objeto do carro para esse campo — senão o script não sabe o que mover.

6. Update() — o coração do script

```
private void Update()
{
    Atualizar(Time.deltaTime);
}
```

Trecho 5 — método Update

- Update() é um método especial do Unity: é chamado **automaticamente uma vez por frame** (por exemplo, 60 vezes por segundo se o jogo rodar a 60 FPS).
- Time.deltaTime: tempo, em segundos, que passou desde o último frame. É essencial usar isso em qualquer cálculo de movimento — senão o jogo ficaria mais rápido ou mais lento dependendo do FPS da máquina que está rodando.
- Aqui o método só chama Atualizar, passando esse tempo adiante.

7. Atualizar() — organizando a ordem das coisas

```
public void Atualizar(float delta)
{
    if (carroTransform == null)
    {
        Debug.LogError("[ControladorCarro] Transform não foi definido!");
        return;
    }
    AtualizarInput(delta);
    AtualizarFisica(delta);
    AplicarMovimento(delta);
}
```

Trecho 6 — método Atualizar

- Primeiro verifica se carroTransform foi preenchido no Inspector. Se foi esquecido, mostra um erro no Console em vez de travar o jogo.
- return; sai do método imediatamente, sem executar o resto.
- Depois chama, **em ordem**, três métodos: ler input → calcular física → mover o objeto. Separar assim deixa o código organizado e mais fácil de entender e depurar (debugar).

8. AtualizarInput() — lendo o jogador

```
entradaDirecao = 0f;
entradaAceleracao = 0f;

string[] joysticks = Input.GetJoystickNames();
bool temControle = joysticks.Length > 0 && !string.IsNullOrEmpty(joysticks[0]);
```

Trecho 7 — detectando controle conectado

- Zera os valores de input a cada frame antes de ler de novo — senão o valor ficaria "grudado" no anterior.
- Input.GetJoystickNames() retorna uma lista com os nomes dos controles conectados ao computador.

- `temControle` verifica se existe pelo menos um controle e se o nome dele não está vazio (às vezes o sistema detecta uma porta vazia, e esse cuidado evita um bug de detecção falsa).

```
if (temControle)
{
    entradaDirecao = Input.GetAxis("Horizontal");
    float rt = Input.GetAxis("RT");
    float lt = Input.GetAxis("LT");
    entradaAceleracao = rt - lt;
}
```

Trecho 8 — input via controle (joystick)

- Se há controle conectado: lê o analógico esquerdo (eixo "Horizontal") para a direção.
- RT e LT representam os gatilhos (triggers) direito e esquerdo do controle. **Atenção:** esses nomes de eixo não existem por padrão no Unity — é preciso criá-los manualmente em *Edit* → *Project Settings* → *Input Manager*.
- `rt - lt`: se só o gatilho direito (acelerar) é apertado, o resultado é positivo; se só o esquerdo (freio/ré), é negativo.

```
else
{
    if (Input.GetKey(KeyCode.A) || Input.GetKey(KeyCode.LeftArrow)) entradaDirecao -= 1;
    if (Input.GetKey(KeyCode.D) || Input.GetKey(KeyCode.RightArrow)) entradaDirecao += 1;
    if (Input.GetKey(KeyCode.W) || Input.GetKey(KeyCode.UpArrow)) entradaAceleracao = 1f;
    if (Input.GetKey(KeyCode.S) || Input.GetKey(KeyCode.DownArrow)) entradaAceleracao = -0.75f;
}
```

Trecho 9 — input via teclado

- Se **não** há controle conectado, o script usa o teclado.
- `Input.GetKey(KeyCode.A)` retorna `true` enquanto a tecla A estiver pressionada — diferente de `GetKeyDown`, que só dispara no instante em que a tecla é apertada.
- Apertar A e D ao mesmo tempo zera o valor ($-1 + 1 = 0$), o que faz sentido: rodas retas.
- A ré (tecla S) tem um fator `-0.75f`, um pouco mais fraco que acelerar para frente (`1f`) — apenas uma escolha de balanceamento do jogo.

```
anguloAlvo = entradaDirecao * anguloMaximoDirecao;
anguloDirecao = Mathf.Lerp(anguloDirecao, anguloAlvo, 10f * delta);
```

Trecho 10 — suavizando o ângulo da direção

- `anguloAlvo` transforma o input (-1 a 1) em um ângulo real (por exemplo, -37° a $+37^\circ$).
- `Mathf.Lerp(a, b, t)` interpola suavemente entre dois valores. Aqui, faz o ângulo das rodas ir **gradualmente** até o alvo, em vez de "pular" instantaneamente — dá a sensação de uma direção mais realista.
- `10f * delta` controla a **velocidade** dessa suavização: quanto maior o valor, mais rápido as rodas respondem ao input.

9. AtualizarFisica() — calculando a velocidade

```
if (entradaAceleracao > 0.1f)
{
    velocidadeAtual += entradaAceleracao * 55f * delta;
}
else if (entradaAceleracao < -0.1f)
{
    velocidadeAtual += entradaAceleracao * 65f * delta;
}
else
{
    velocidadeAtual *= 0.935f;
}

velocidadeAtual = Mathf.Clamp(velocidadeAtual, -velocidadeMaximaRe, velocidadeMaxima);
```

Trecho 11 — cálculo de aceleração, ré e atrito

- Se está acelerando para frente (acima de 0.1, uma "zona morta" para evitar tremulação de input): aumenta a velocidade, multiplicando por 55f (fator de aceleração) e por delta (para ser independente do FPS).
- Se está acelerando para trás (abaixo de -0.1): mesma lógica, mas com fator 65f.
- Se **não** está apertando nada, `velocidadeAtual *= 0.935f` é o **atrito/desaceleração natural**: a cada frame, a velocidade perde 6,5% do valor, fazendo o carro desacelerar suavemente até parar (como um "freio motor").
- `Mathf.Clamp(valor, min, max)` garante que a velocidade nunca ultrapasse os limites definidos (`velocidadeMaximaRe` e `velocidadeMaxima`).

10. AplicarMovimento() — movendo e girando o carro

```
float yaw = carroTransform.eulerAngles.y;
Vector3 direcao = Quaternion.Euler(0, yaw, 0) * Vector3.back;
```

Trecho 12 — calculando a direção da frente do carro

- `eulerAngles.y` pega a rotação atual do carro no eixo Y — o eixo "vertical", que controla para onde o carro está virado no plano horizontal.
- `Quaternion.Euler(0, yaw, 0) * Vector3.back` pega o vetor "para trás" (`Vector3.back` = (0,0,-1)) e o rotaciona de acordo com o yaw do carro. O resultado é um vetor que aponta exatamente para a frente do carro, considerando a rotação atual dele. Foi usado `Vector3.back` em vez de `Vector3.forward` provavelmente por causa da orientação do modelo 3D importado — pode ser necessário ajustar dependendo do seu modelo.

```
float distancia = velocidadeAtual * delta * 14f;
Vector3 deslocamento = direcao * distancia;
carroTransform.position += deslocamento;
```

Trecho 13 — aplicando o deslocamento

- `distancia` é o quanto o carro deve se mover nesse frame, com base na velocidade atual. O 14f é apenas um multiplicador de escala, que ajusta a "sensação" de velocidade visualmente.
- `deslocamento` é o vetor direção multiplicado pela distância — ou seja, para onde e quanto mover.

- `carroTransform.position += deslocamento;` move o carro de fato, somando o deslocamento à posição atual.

```
if (Mathf.Abs(velocidadeAtual) > 3f)
{
    float fatorRe = velocidadeAtual > 0 ? 1f : -0.5f;
    carroTransform.Rotate(Vector3.up, anguloDirecao * fatorRe * delta * 4.8f);
}
```

Trecho 14 — girando o carro

- Só permite o carro **virar** se a velocidade estiver acima de 3f — evita que o carro gire "no lugar" quando está parado, o que é mais realista.
- `fatorRe` usa um operador ternário (`condição ? valorSeVerdadeiro : valorSeFalso`). Indo para frente, gira normalmente (1f); de ré, gira na metade da intensidade e invertido (-0.5f) — imitando como carros reais viram diferente quando estão dando marcha à ré.
- `carroTransform.Rotate(Vector3.up, ...)` gira o objeto em torno do eixo Y (vertical) pela quantidade calculada.

11. Getters — expondo dados para outros scripts

```
public float GetAnguloDirecao() => anguloDirecao;  
public float GetVelocidadeAtual() => velocidadeAtual;
```

Trecho 15 — métodos de acesso (getters)

- => é a "expression body syntax": uma forma curta de escrever métodos que só retornam um valor, sem precisar da sintaxe completa { return ...; }.
- Esses métodos permitem que **outros scripts** — por exemplo, um script de UI mostrando o velocímetro, ou um script de áudio do motor — leiam esses valores sem poder alterá-los diretamente, já que as variáveis originais são `private`.

12. Glossário rápido de termos do Unity/C#

MonoBehaviour — Classe base do Unity que dá a um script acesso ao ciclo de vida do jogo (Update, Start, etc.) e permite anexá-lo a um GameObject.

GameObject — Qualquer objeto presente em uma cena do Unity (o carro, uma luz, uma câmera etc.).

Transform — Componente presente em todo GameObject; guarda posição, rotação e escala.

Componente — Um "pedaço" de funcionalidade anexado a um GameObject (scripts, colliders, luzes, etc.).

Inspector — Painel do Unity onde se veem e editam os valores dos componentes de um objeto selecionado.

float — Tipo de dado numérico com casas decimais (ponto flutuante).

delta / Time.deltaTime — Tempo, em segundos, decorrido desde o último frame — usado para que o movimento seja independente da taxa de quadros (FPS).

Lerp (interpolação linear) — Função que calcula um valor intermediário suave entre dois pontos, dado um percentual de progresso.

Clamp — Função que "trava" um valor dentro de um intervalo mínimo e máximo.

Quaternion — Estrutura matemática usada pelo Unity para representar rotações de forma eficiente, evitando problemas como o gimbal lock.

Dica: para configurar os eixos RT e LT do controle, acesse *Edit* → *Project Settings* → *Input Manager* no Unity e crie os eixos manualmente antes de testar o suporte a joystick.